

Với điều kiện ghi rõ nguồn gốc, Google cho phép sao chép các bảng và hình ảnh trong bài báo này chỉ để sử dụng cho mục đích báo chí hoặc các công trình học thuật.

Chỉ cần sự chú ý là đủ.

Ashish Vaswani*
Bộ não Google
avaswani@google.com

Noam Shazeer*
Bộ não Google
noam@google.com

Niki Parmar*
Nghiên cứu của Google
nikip@google.com

Jakob Uszkoreit*
Nghiên cứu của Google
usz@google.com

Lion Jones*
Nghiên cứu của Google
llion@google.com

Aidan N. Gomez*[†]
Đại học Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Bộ não Google
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Tóm tắt

Các mô hình chuyển đổi chuỗi chiếm ưu thế hiện nay dựa trên các mạng nơ-ron hồi quy hoặc tích chập phức tạp bao gồm bộ mã hóa và bộ giải mã. Các mô hình hoạt động tốt nhất cũng kết nối bộ mã hóa và bộ giải mã thông qua cơ chế chú ý. Chúng tôi đề xuất một kiến trúc mạng đơn giản mới, Transformer, chỉ dựa trên cơ chế chú ý, loại bỏ hoàn toàn hồi quy và tích chập. Các thí nghiệm trên hai nhiệm vụ dịch máy cho thấy các mô hình này vượt trội về chất lượng, đồng thời có khả năng song song hóa cao hơn và yêu cầu thời gian huấn luyện ít hơn đáng kể. Mô hình của chúng tôi đạt được 28,4 BLEU trên nhiệm vụ dịch tiếng Anh sang tiếng Đức WMT 2014, cải thiện hơn 2 BLEU so với các kết quả tốt nhất hiện có, bao gồm cả các mô hình kết hợp. Trên nhiệm vụ dịch tiếng Anh sang tiếng Pháp WMT 2014, mô hình của chúng tôi thiết lập điểm BLEU mới tốt nhất hiện nay cho một mô hình duy nhất là 41,8 sau khi huấn luyện trong 3,5 ngày trên tám GPU, một phần nhỏ chi phí huấn luyện so với các mô hình tốt nhất trong tài liệu. Chúng tôi chứng minh rằng Transformer có khả năng khái quát hóa tốt cho các nhiệm vụ khác bằng cách áp dụng thành công nó vào việc phân tích cú pháp thành phần câu tiếng Anh với cả dữ liệu huấn luyện lớn và hạn chế.

*Đóng góp ngang nhau. Thứ tự liệt kê ngẫu nhiên. Jakob đề xuất thay thế mạng RNN bằng cơ chế tự chú ý và bắt đầu nỗ lực đánh giá ý tưởng này. Ashish, cùng với Illia, đã thiết kế và triển khai các mô hình Transformer đầu tiên và đóng vai trò quan trọng trong mọi khía cạnh của công việc này. Noam đề xuất cơ chế chú ý tích vô hướng có tỷ lệ, chú ý đa đầu và biểu diễn vị trí không tham số và trở thành người còn lại tham gia vào hầu hết mọi chi tiết. Niki đã thiết kế, triển khai, tinh chỉnh và đánh giá vô số biến thể mô hình trong mã nguồn gốc của chúng tôi và tensor2tensor. Llion cũng đã thử nghiệm các biến thể mô hình mới, chịu trách nhiệm về mã nguồn ban đầu của chúng tôi, cũng như suy luận và trực quan hóa hiệu quả. Łukasz và Aidan đã dành vô số ngày dài để thiết kế các phần khác nhau và triển khai tensor2tensor, thay thế mã nguồn trước đó của chúng tôi, cải thiện đáng kể kết quả và đẩy nhanh đáng kể nghiên cứu của chúng tôi.

[†]Các công việc đã thực hiện trong thời gian làm việc tại Google Brain.

[‡]Các công việc đã thực hiện trong thời gian làm việc tại Google Research.

1. Giới thiệu

Mạng nơ-ron hồi quy, đặc biệt là mạng nơ-ron hồi quy có cổng [7], đã được khẳng định là các phương pháp tiên tiến nhất trong các vấn đề mô hình hóa chuỗi và chuyển đổi như mô hình hóa ngôn ngữ và dịch máy [35, 2, 5]. Nhiều nỗ lực kể từ đó đã tiếp tục thúc đẩy ranh giới của các mô hình ngôn ngữ hồi quy và kiến trúc mã hóa-giải mã [38, 24, 15].

Các mô hình mạng nơ-ron hồi quy thường phân tích quá trình tính toán dựa trên vị trí của các ký hiệu trong chuỗi đầu vào và đầu ra. Bằng cách căn chỉnh các vị trí này với các bước trong thời gian tính toán, chúng tạo ra một chuỗi các trạng thái ẩn h_t , như một hàm của trạng thái ẩn trước đó h_{t-1} và đầu vào cho vị trí t . Bản chất tuần tự vốn có này ngăn cản việc song song hóa trong các ví dụ huấn luyện, điều này trở nên quan trọng ở độ dài chuỗi dài hơn, vì các hạn chế về bộ nhớ giới hạn việc gom nhóm các ví dụ. Công việc gần đây đã đạt được những cải tiến đáng kể về hiệu quả tính toán thông qua các thủ thuật phân tích nhân tử [21] và tính toán có điều kiện [32], đồng thời cũng cải thiện hiệu suất mô hình trong trường hợp sau. Tuy nhiên, hạn chế cơ bản của tính toán tuần tự vẫn còn tồn tại.

Các cơ chế chú ý đã trở thành một phần không thể thiếu của các mô hình trình tự và chuyển đổi hấp dẫn trong nhiều nhiệm vụ, cho phép mô hình hóa các mối phụ thuộc mà không cần quan tâm đến khoảng cách của chúng trong chuỗi đầu vào hoặc đầu ra [2, 19]. Tuy nhiên, trong hầu hết các trường hợp [27], các cơ chế chú ý như vậy được sử dụng kết hợp với mạng nơ-ron hồi quy.

Trong công trình này, chúng tôi đề xuất Transformer, một kiến trúc mô hình loại bỏ tính lặp lại và thay vào đó hoàn toàn dựa vào cơ chế chú ý để xác định các mối quan hệ toàn cục giữa đầu vào và đầu ra. Transformer cho phép song song hóa đáng kể và có thể đạt được hiệu suất dịch thuật vượt trội sau khi được huấn luyện chỉ trong vòng mười hai giờ trên tám GPU P100.

2. Bối cảnh

Mục tiêu giảm thiểu tính toán tuần tự cũng là nền tảng của Extended Neural GPU [16], ByteNet [18] và ConvS2S [9], tất cả đều sử dụng mạng nơ-ron tích chập làm khối xây dựng cơ bản, tính toán các biểu diễn ẩn song song cho tất cả các vị trí đầu vào và đầu ra. Trong các mô hình này, số lượng phép toán cần thiết để liên hệ các tín hiệu từ hai vị trí đầu vào hoặc đầu ra bất kỳ tăng theo khoảng cách giữa các vị trí, tuyến tính đối với ConvS2S và logarit đối với ByteNet. Điều này làm cho việc học các mối quan hệ phụ thuộc giữa các vị trí xa nhau trở nên khó khăn hơn [12]. Trong Transformer, điều này được giảm xuống một số phép toán không đổi, mặc dù phải trả giá bằng độ phân giải hiệu quả giảm do trung bình hóa các vị trí được trọng số hóa theo sự chú ý, một hiệu ứng mà chúng tôi khắc phục bằng Multi-Head Attention như được mô tả trong phần 3.2.

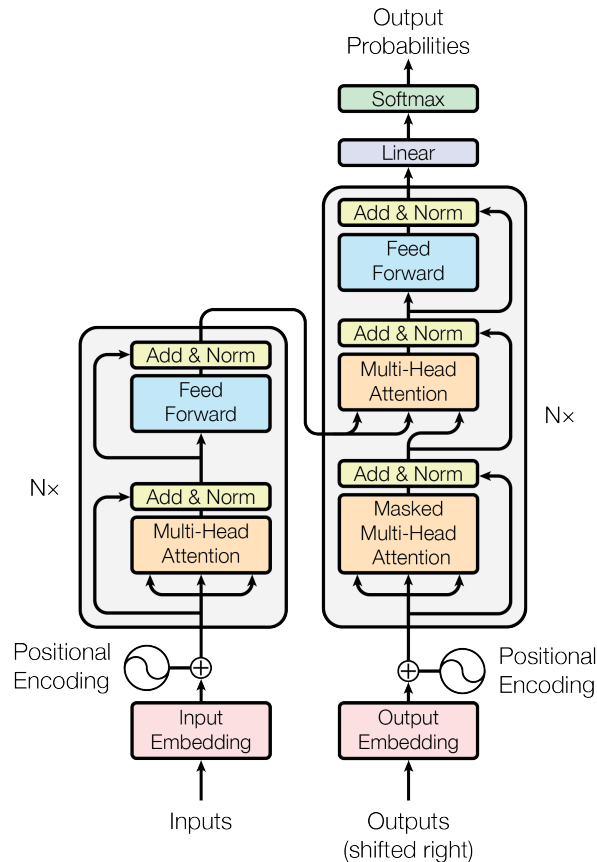
Tự chú ý, đôi khi được gọi là chú ý nội tại, là một cơ chế chú ý liên quan đến các vị trí khác nhau của một chuỗi duy nhất để tính toán biểu diễn của chuỗi đó. Tự chú ý đã được sử dụng thành công trong nhiều nhiệm vụ khác nhau bao gồm đọc hiểu, tóm tắt trừu tượng, suy luận văn bản và học các biểu diễn câu đọc lập với nhiệm vụ [4, 27, 28, 22].

Mạng bộ nhớ đầu cuối dựa trên cơ chế chú ý lặp lại thay vì lặp lại theo trình tự và đã được chứng minh là hoạt động tốt trong các nhiệm vụ trả lời câu hỏi ngôn ngữ đơn giản và mô hình hóa ngôn ngữ [34].

Tuy nhiên, theo hiểu biết của chúng tôi, Transformer là mô hình chuyển đổi đầu tiên dựa hoàn toàn vào cơ chế tự chú ý để tính toán biểu diễn đầu vào và đầu ra mà không sử dụng RNN được căn chỉnh theo trình tự hoặc phép tích chập. Trong các phần tiếp theo, chúng tôi sẽ mô tả Transformer, giải thích cơ chế tự chú ý và thảo luận về những ưu điểm của nó so với các mô hình như [17, 18] và [9].

3. Kiến trúc mô hình

Hầu hết các mô hình chuyển đổi chuỗi thần kinh cạnh tranh đều có cấu trúc bộ mã hóa-bộ giải mã [5, 2, 35]. Ở đây, bộ mã hóa ánh xạ một chuỗi đầu vào của các biểu diễn ký hiệu ($x_1, \dots, x_n$) thành một chuỗi các biểu diễn liên tục $z = (z_1, \dots, z_n)$. Được cho z , Sau đó, bộ giải mã tạo ra một chuỗi đầu ra ($y_1, \dots, y_m$) của các ký hiệu từng phần tử một. Ở mỗi bước, mô hình là tự hồi quy [10], sử dụng các ký hiệu được tạo ra trước đó làm đầu vào bổ sung khi tạo ra ký hiệu tiếp theo.



Hình 1: Mô hình kiến trúc Transformer.

Mô hình Transformer tuân theo kiến trúc tổng thể này bằng cách sử dụng các lớp tự chú ý xếp chồng lên nhau và các lớp kết nối đầy đủ theo từng điểm cho cả bộ mã hóa và bộ giải mã, được thể hiện lần lượt ở nửa bên trái và bên phải của Hình 1.

3.1 Ngăn xếp mã hóa và giải mã

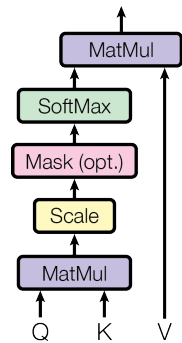
Bộ mã hóa: Bộ mã hóa được cấu tạo từ một chồng các phần tử $N=6$ các lớp giống hệt nhau. Mỗi lớp có hai lớp con. Lớp đầu tiên là cơ chế tự chú ý đa đầu, và lớp thứ hai là mạng truyền thẳng đơn giản, kết nối đầy đủ theo vị trí. Chúng tôi sử dụng kết nối dư [11] xung quanh mỗi trong hai lớp con, tiếp theo là chuẩn hóa lớp [1]. Nghĩa là, đầu ra của mỗi lớp con là $\text{LayerNorm}(x + \text{Lớp con}(x))$, ở đây $\text{Lớp con}(x)$ là hàm được thực hiện bởi chính lớp con. Để tạo điều kiện thuận lợi cho các kết nối dư này, tất cả các lớp con trong mô hình, cũng như các lớp nhúng, đều tạo ra đầu ra có kích thước $d_{\text{người mẫu}} = 512$.

Bộ giải mã: Bộ giải mã cũng bao gồm một chồng các $N=6$ các lớp giống hệt nhau. Ngoài hai lớp con trong mỗi lớp mã hóa, bộ giải mã chèn thêm một lớp con thứ ba, thực hiện cơ chế chú ý đa đầu trên đầu ra của chồng mã hóa. Tương tự như bộ mã hóa, chúng tôi sử dụng các kết nối dư xung quanh mỗi lớp con, tiếp theo là chuẩn hóa lớp. Chúng tôi cũng sửa đổi lớp con tự chú ý trong chồng giải mã để ngăn các vị trí chú ý đến các vị trí tiếp theo. Việc che giấu này, kết hợp với thực tế là các nhúng đầu ra được dịch chuyển một vị trí, đảm bảo rằng các dự đoán cho vị trí T chỉ có thể dựa vào các kết quả đầu ra đã biết tại các vị trí nhỏ hơn T .

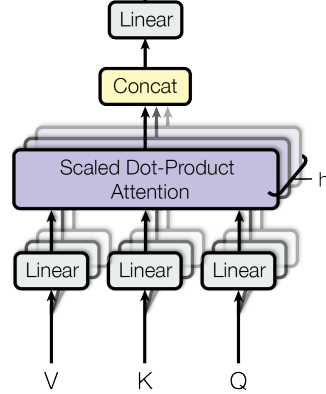
3.2 Sự chú ý

Hàm chú ý có thể được mô tả như một phép ánh xạ truy vấn và một tập hợp các cặp khóa-giá trị đến một đầu ra, trong đó truy vấn, khóa, giá trị và đầu ra đều là các vector. Đầu ra được tính toán như một tổng có trọng số.

Chú ý tích vô hướng được điều chỉnh



Sự chú ý đa đầu



Hình 2: (trái) Cơ chế chú ý tích vô hướng được điều chỉnh tỷ lệ. (phải) Cơ chế chú ý đa đầu bao gồm nhiều lớp chú ý chạy song song.

trong đó trọng số được gán cho mỗi giá trị được tính toán bằng một hàm tương thích của truy vấn với khóa tương ứng.

3.2.1 Chú ý tích vô hướng được điều chỉnh

Chúng tôi gọi cơ chế chú ý đặc biệt này là "Cơ chế chú ý tích vô hướng có tỷ lệ" (Hình 2). Đầu vào bao gồm các truy vấn và khóa có kích thước d_k . Một giá trị và kích thước d_v . Chúng ta tính tích vô hướng của... truy vấn với tất cả các khóa, chia mỗi khóa cho d_k và áp dụng hàm softmax để thu được trọng số trên các giá trị.

Trên thực tế, chúng ta tính toán hàm chú ý trên một tập hợp các truy vấn đồng thời, được đóng gói lại thành một ma trận. Các khóa và giá trị cũng được đóng gói cùng nhau thành các ma trận. K và V . Chúng ta tính toán ma trận đầu ra như sau:

$$\text{Chú ý}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{d_k}\right)V \quad (1)$$

Hai hàm chú ý được sử dụng phổ biến nhất là chú ý cộng [2] và chú ý tích vô hướng (nhân). Chú ý tích vô hướng giống hệt thuật toán của chúng tôi, ngoại trừ hệ số tỷ lệ của $\frac{1}{d_k}$. Cơ chế chú ý cộng tính toán hàm tương thích bằng cách sử dụng mạng nơ-ron truyền thẳng với một lớp ẩn duy nhất. Mặc dù cả hai có độ phức tạp lý thuyết tương tự nhau, nhưng cơ chế attention tích vô hướng nhanh hơn và tiết kiệm không gian hơn nhiều trong thực tế, vì nó có thể được triển khai bằng mã nhân ma trận được tối ưu hóa cao.

Trong khi đối với các giá trị nhỏ của d_k Hai cơ chế này hoạt động tương tự nhau, cơ chế chú ý cộng (additive attention) hoạt động tốt hơn cơ chế chú ý tích vô hướng (dot product attention) mà không cần điều chỉnh tỷ lệ đối với các giá trị lớn hơn của d_k [3]. Chúng tôi nghi ngờ rằng đối với các giá trị lớn của d_k Các tích vô hướng tăng lên đáng kể về độ lớn, đẩy hàm softmax vào các vùng có độ dốc cực nhỏ. Để khắc phục hiệu ứng này, chúng ta nhân các tích vô hướng với $\frac{1}{d_k}$.

3.2.2 Sự chú ý đa đầu

Thay vì thực hiện một chức năng chú ý duy nhất với $d_{\text{người mẫu}}$ Chúng tôi nhận thấy việc chiếu tuyến tính các truy vấn, khóa và giá trị có nhiều chiều là hữu ích. Nhiều lần với các phép chiếu tuyến tính khác nhau đã được học. d_k, d_v và $d_{\text{tương ứng}}$ với các chiều. Trên mỗi phiên bản được chiếu này của các truy vấn, khóa và giá trị, chúng ta thực hiện chức năng chú ý song song, tạo ra kết quả như sau: d_v chiều

Để minh họa lý do tại sao tích vô hướng lại lớn, giả sử các thành phần của q và k là ngẫu nhiên độc lập các biến có giá trị trung bình 0 và phương sai 1. Sau đó là tích vô hướng của chúng, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ có nghĩa là 0 và phương sai d_k .

các giá trị đầu ra. Chúng được nối lại với nhau và chiếu một lần nữa, tạo ra các giá trị cuối cùng, như được minh họa trong Hình 2.

Cơ chế chú ý đa đầu cho phép mô hình cùng lúc chú ý đến thông tin từ các không gian con biểu diễn khác nhau ở các vị trí khác nhau. Với một đầu chú ý đơn, việc lấy trung bình sẽ cản trở điều này.

$$\text{Đa đầu } (Q, K, V) = \text{Nối}(\text{đầu}_1, \dots, \text{cái đầu}_h) W_O$$

$$\text{Ở đâu cái đầu}_{T\text{đi}} = \text{Chú ý}(QW_{QT\text{đi}}, KW_{KT\text{đi}}, VW_{VT\text{đi}})$$

Trong đó các phép chiếu là các ma trận tham số. $W_Q, W_{T\text{đi}} \in \mathbb{R}^{d_{\text{người mẫu}} \times d_k}$, $W_K, W_{T\text{đi}} \in \mathbb{R}^{d_{\text{người mẫu}} \times d_k}$, $W_V, W_{T\text{đi}} \in \mathbb{R}^{d_{\text{người mẫu}} \times d_v}$
 Và $W_O \in \mathbb{R}^{d_{\text{phân giải cao}} \times d_{\text{người mẫu}}}$.

Trong công trình này, chúng tôi sử dụng $h=8$ Các lớp chú ý song song, hay còn gọi là các đầu chú ý. Đối với mỗi lớp này, chúng ta sử dụng $d_k=d_v=d_{\text{người mẫu}}/h=64$. Do kích thước của mỗi đầu được giảm bớt, tổng chi phí tính toán tương tự như cơ chế chú ý một đầu với kích thước đầy đủ.

3.2.3 Ứng dụng của cơ chế Attention trong mô hình của chúng tôi

Bộ chuyển đổi (Transformer) sử dụng cơ chế chú ý đa đầu theo ba cách khác nhau:

- Trong các lớp "chú ý bộ mã hóa-bộ giải mã", các truy vấn đến từ lớp giải mã trước đó và các khóa và giá trị bộ nhớ đến từ đầu ra của bộ mã hóa. Điều này cho phép mọi vị trí trong bộ giải mã chú ý đến tất cả các vị trí trong chuỗi đầu vào. Điều này mô phỏng các cơ chế chú ý bộ mã hóa-bộ giải mã điển hình trong các mô hình chuỗi-đến-chuỗi như [38, 2, 9].
- Bộ mã hóa chứa các lớp tự chú ý. Trong một lớp tự chú ý, tất cả các khóa, giá trị và truy vấn đều đến từ cùng một vị trí, trong trường hợp này là đầu ra của lớp trước đó trong bộ mã hóa. Mỗi vị trí trong bộ mã hóa có thể chú ý đến tất cả các vị trí trong lớp trước đó của bộ mã hóa.
- Tương tự, các lớp tự chú ý trong bộ giải mã cho phép mỗi vị trí trong bộ giải mã chú ý đến tất cả các vị trí trong bộ giải mã cho đến và bao gồm cả vị trí đó. Chúng ta cần ngăn chặn luồng thông tin hướng sang trái trong bộ giải mã để duy trì tính chất tự hồi quy. Chúng ta thực hiện điều này bên trong cơ chế chú ý tích vô hướng được điều chỉnh bằng cách che mặt nạ (đặt thành $-\infty$) tất cả các giá trị trong đầu vào của hàm softmax tương ứng với các kết nối bất hợp lệ. Xem Hình 2.

3.3 Mạng nơ-ron truyền thẳng theo vị trí

Ngoài các lớp con xử lý sự chú ý, mỗi lớp trong bộ mã hóa và giải mã của chúng tôi đều chứa một mạng nơ-ron truyền thẳng kết nối đầy đủ, được áp dụng riêng biệt và giống hệt nhau cho từng vị trí. Mạng này bao gồm hai phép biến đổi tuyến tính với hàm kích hoạt ReLU ở giữa.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

Mặc dù các phép biến đổi tuyến tính giống nhau ở các vị trí khác nhau, nhưng chúng sử dụng các tham số khác nhau giữa các lớp. Một cách khác để mô tả điều này là hai phép tích chập với kích thước kernel là 1. Kích thước của đầu vào và đầu ra là $d_{\text{người mẫu}}=512$, và lớp bên trong có chiều không gian $d_{\text{ffn}}=2048$.

3.4 Các phép nhúng và hàm Softmax

Tương tự như các mô hình chuyển đổi chuỗi khác, chúng tôi sử dụng các embedding đã học để chuyển đổi các token đầu vào và token đầu ra thành các vector có kích thước $d_{\text{người mẫu}}$. Chúng tôi cũng sử dụng phép biến đổi tuyến tính đã học thông thường và hàm softmax để chuyển đổi đầu ra của bộ giải mã thành xác suất dự đoán token tiếp theo. Trong mô hình của chúng tôi, chúng tôi chia sẻ cùng một ma trận trọng số giữa hai lớp nhúng và lớp tiền xử lý. $\sqrt{\text{softmax}}$ biến đổi tuyến tính, tương tự như [30]. Trong các lớp nhúng, chúng tôi nhân các trọng số đó với $d_{\text{người mẫu}}$.

Bảng 1: Chiều dài đường dẫn tối đa, độ phức tạp trên mỗi lớp và số lượng thao tác tuần tự tối thiểu cho các loại lớp khác nhau. Mà độ dài của chuỗi, là chiều biểu diễn, là kích thước hạt nhân của phép tích chập và kích thước của khu phố trong phạm vi tự chú ý hạn chế.

Loại lớp	Độ phức tạp trên mỗi lớp	Tuần tự Hoạt động	Chiều dài đường dẫn tối đa
Sự tự chú ý	$O(N_2 \cdot d)$	$O(1)$	$O(1)$
Tái phát	$O(N \cdot d_2)$	$O(N)$	$O(N)$
Tích chập	$O(k \cdot N \cdot d_2)$	$O(1)$	$O(\text{nhật ký}(N))$
Tự chú ý (có giới hạn)	$r \cdot N \cdot d$	$O(1)$	$O(n/r)$

3.5 Mã hóa vị trí

Vì mô hình của chúng ta không chứa phép lặp lại hay phép tích chập, để mô hình có thể sử dụng thứ tự của chuỗi, chúng ta phải đưa vào một số thông tin về vị trí tương đối hoặc tuyệt đối của các token trong chuỗi. Để làm được điều này, chúng ta thêm "mã hóa vị trí" vào các embedding đầu vào ở cuối các ngăn xếp mã hóa và giải mã. Các mã hóa vị trí có cùng kích thước. ^{đang chờ mẫu} như các phần nhúng, để hai phần có thể được cộng lại. Có nhiều lựa chọn mã hóa vị trí, được học và cố định [9].

Trong nghiên cứu này, chúng tôi sử dụng các hàm sin và cosin với các tần số khác nhau:

$$\text{Thế dụ } \sin(\text{vị trí} / 10000^{2k}) = \sin(\text{vị trí} / 10000^{2k} \cdot \text{nhận dạng người mẫu})$$

$$\text{Thế dụ } \cos(\text{vị trí} / 10000^{2k+1}) = \cos(\text{vị trí} / 10000^{2k+1} \cdot \text{nhận dạng người mẫu})$$

Ở đây vị trí là vị trí và $2k$ là chiều. Tức là, mỗi chiều của mã hóa vị trí tương ứng với một sóng hình sin. Bước sóng tạo thành một cặp số nhân hình học từ 2π ĐẾN $10000 \cdot 2\pi$. Chúng tôi chọn hàm này vì chúng tôi giả định rằng nó sẽ cho phép mô hình dễ dàng học cách chú ý theo vị trí tương đối, vì với bất kỳ độ lệch cố định nào k , $\text{Thế dụ } \sin(\text{vị trí} + k)$ có thể được biểu diễn dưới dạng hàm tuyến tính của

$\text{Thế dụ } \sin(\text{vị trí})$.

Chúng tôi cũng đã thử nghiệm sử dụng nhúng vị trí đã học [9] thay thế và thấy rằng hai phiên bản tạo ra kết quả gần như giống hệt nhau (xem Bảng 3 hàng (E)). Chúng tôi đã chọn phiên bản hình sin vì nó có thể cho phép mô hình ngoại suy đến độ dài chuỗi dài hơn những độ dài gặp phải trong quá trình huấn luyện.

4. Tại sao cần sự chú ý đến bản thân?

Trong phần này, chúng ta sẽ so sánh các khía cạnh khác nhau của lớp tự chú ý với các lớp mạng nơ-ron hồi quy và mạng nơ-ron tích chập thường được sử dụng để ánh xạ một chuỗi biểu diễn ký hiệu có độ dài thay đổi. $(x_1, \dots, x_N)$ thành một chuỗi khác có độ dài bằng nhau $(z_1, \dots, z_N)$, với $x, z \in \mathbb{R}^N$. Ví dụ như một lớp ẩn trong bộ mã hóa hoặc giải mã chuyển đổi chuỗi điển hình. Để thúc đẩy việc sử dụng cơ chế tự chú ý, chúng tôi xem xét ba yếu tố mong muốn.

Một trong những yếu tố đó là độ phức tạp tính toán tổng thể trên mỗi lớp. Yếu tố khác là lượng tính toán có thể được song song hóa, được đo bằng số lượng tối thiểu các thao tác tuần tự cần thiết.

Thứ ba là độ dài đường dẫn giữa các phụ thuộc tầm xa trong mạng. Học các phụ thuộc tầm xa là một thách thức quan trọng trong nhiều nhiệm vụ chuyển đổi chuỗi. Một yếu tố quan trọng ảnh hưởng đến khả năng học các phụ thuộc như vậy là độ dài của các đường dẫn mà tín hiệu tiến và lùi phải đi qua trong mạng. Các đường dẫn này càng ngắn giữa bất kỳ kết hợp nào của các vị trí trong chuỗi đầu vào và đầu ra thì việc học các phụ thuộc tầm xa càng dễ dàng hơn [12]. Do đó, chúng tôi cũng so sánh độ dài đường dẫn tối đa giữa bất kỳ hai vị trí đầu vào và đầu ra nào trong các mạng được cấu tạo từ các loại lớp khác nhau.

Như đã nêu trong Bảng 1, lớp tự chú ý kết nối tất cả các vị trí với một số lượng thao tác được thực hiện tuần tự không đổi, trong khi lớp mạng nơ-ron hồi quy yêu cầu $O(N)$ các thao tác tuần tự. Về độ phức tạp tính toán, các lớp tự chú ý nhanh hơn các lớp mạng nơ-ron hồi quy khi chuỗi

chiều dài N nhỏ hơn chiều biểu diễn d , điều này thường xảy ra nhất với các biểu diễn câu được sử dụng bởi các mô hình tiên tiến nhất trong dịch máy, chẳng hạn như biểu diễn word-piece [38] và byte-pair [31]. Để cải thiện hiệu suất tính toán cho các tác vụ liên quan đến chuỗi rất dài, cơ chế tự chú ý có thể được giới hạn chỉ xem xét một vùng lân cận có kích thước k trong chuỗi đầu vào tập trung xung quanh vị trí đầu ra tương ứng. Điều này sẽ làm tăng chiều dài đường dẫn tối đa lên $O(n/k)$. Chúng tôi dự định sẽ nghiên cứu sâu hơn phương pháp này trong các công trình nghiên cứu tiếp theo.

Một lớp tích chập đơn với độ rộng kernel $k < n$ không kết nối tất cả các cặp vị trí đầu vào và đầu ra. Việc này đòi hỏi một ngăn xếp. $O(n/k)$ các lớp tích chập trong trường hợp các kernel liên kề, hoặc $O(n \log(M))$ trong trường hợp tích chập giãn nở [18], làm tăng chiều dài của các đường dẫn dài nhất giữa bất kỳ hai vị trí nào trong mạng. Các lớp tích chập thường tốn kém hơn các lớp lặp lại, với hệ số k Tuy nhiên, các phép chập tách rời [6] làm giảm đáng kể độ phức tạp, thành $O(k \cdot N \cdot d \cdot N \cdot d)$. Ngay cả với $k=N$ Tuy nhiên, độ phức tạp của phép tích chập tách rời bằng với sự kết hợp của lớp tự chú ý và lớp truyền thẳng từng điểm, phương pháp mà chúng tôi sử dụng trong mô hình của mình.

Một lợi ích phụ là khả năng tự chú ý có thể tạo ra các mô hình dễ hiểu hơn. Chúng tôi kiểm tra sự phân bố chú ý từ các mô hình của mình và trình bày cũng như thảo luận các ví dụ trong phụ lục. Không chỉ các đầu chú ý riêng lẻ học cách thực hiện các nhiệm vụ khác nhau một cách rõ ràng, mà nhiều đầu chú ý còn thể hiện hành vi liên quan đến cấu trúc cú pháp và ngữ nghĩa của câu.

5. Đào tạo

Phần này mô tả chế độ huấn luyện cho các mô hình của chúng tôi.

5.1 Dữ liệu huấn luyện và xử lý theo lô

Chúng tôi đã huấn luyện trên bộ dữ liệu WMT 2014 Anh-Đức tiêu chuẩn bao gồm khoảng 4,5 triệu cặp câu. Các câu được mã hóa bằng cách sử dụng mã hóa cặp byte [3], có từ vựng nguồn-mục tiêu chung gồm khoảng 37000 token. Đối với Anh-Pháp, chúng tôi đã sử dụng bộ dữ liệu WMT 2014 Anh-Pháp lớn hơn đáng kể bao gồm 36 triệu câu và chia token thành từ vựng gồm 32000 từ [38]. Các cặp câu được nhóm lại với nhau theo độ dài chuỗi gần đúng. Mỗi lô huấn luyện chứa một tập hợp các cặp câu chứa khoảng 25000 token nguồn và 25000 token mục tiêu.

5.2 Phần cứng và Lịch trình

Chúng tôi đã huấn luyện các mô hình của mình trên một máy tính với 8 GPU NVIDIA P100. Đối với các mô hình cơ bản sử dụng các siêu tham số được mô tả trong toàn bộ bài báo, mỗi bước huấn luyện mất khoảng 0,4 giây. Chúng tôi đã huấn luyện các mô hình cơ bản tổng cộng 100.000 bước, tương đương 12 giờ. Đối với các mô hình lớn (được mô tả ở dòng cuối cùng của bảng 3), thời gian mỗi bước là 1,0 giây. Các mô hình lớn được huấn luyện trong 300.000 bước (3,5 ngày).

5.3 Trình tối ưu hóa

Chúng tôi đã sử dụng trình tối ưu hóa Adam [20] với $\beta_1=0.9, \beta_2=0.98$ và $\epsilon=10^{-9}$. Chúng tôi đã thay đổi tốc độ học tập trong suốt quá trình huấn luyện, theo công thức:

$$\text{tỷ lệ lớn} = \frac{1}{\sqrt{\text{bước chân số} + 0.5}} \quad (\text{bước chân số khởi động các bước} = 1.5) \quad (3)$$

Điều này tương ứng với việc tăng tốc độ học một cách tuyến tính trong giai đoạn đầu tiên. khởi động các bước các bước huấn luyện, và sau đó giảm dần theo tỷ lệ nghịch với căn bậc hai của số bước. Chúng tôi đã sử dụng khởi động các bước = 4000.

5.4 Chuẩn hóa

Chúng tôi sử dụng ba loại chuẩn hóa trong quá trình huấn luyện:

Bảng 2: Mô hình Transformer đạt điểm BLEU tốt hơn so với các mô hình tiên tiến trước đây trên các bài kiểm tra newstest2014 từ tiếng Anh sang tiếng Đức và tiếng Anh sang tiếng Pháp với chi phí huấn luyện thấp hơn đáng kể.

Người mẫu	BLEU		Chi phí đào tạo (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23,75			
Deep-Att + PosUnk [39]		39,2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24,6	39,92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25,16	40,46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
Bộ Giáo dục [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Tập hợp Deep-Att + PosUnk [39]		40,4		$8.0 \cdot 10^{20}$
Tập hợp GNMT + RL [38] Tập hợp	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S [9]	26,36	41,29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Máy biến áp (mẫu cơ bản)	27.3	38.1	$3.3 \cdot 10^{18}$	
Máy biến áp (mẫu lớn)	28,4	41,8	$2.3 \cdot 10^{19}$	

Bỏ học dưChúng tôi áp dụng dropout [33] cho đầu ra của mỗi lớp con, trước khi nó được thêm vào đầu vào của lớp con và được chuẩn hóa. Ngoài ra, chúng tôi áp dụng dropout cho tổng của các embedding và mã hóa vị trí trong cả ngăn xếp bộ mã hóa và bộ giải mã. Đối với mô hình cơ sở, chúng tôi sử dụng tỷ lệ $P_{\text{làm rơi}}=0.1$.

Làm mịn nhânTrong quá trình huấn luyện, chúng tôi đã sử dụng phương pháp làm mịn nhân giá trị $\epsilon_{ls}=0.1$ [36]. Điều này làm giảm sự bối rối, vì mô hình học cách trở nên không chắc chắn hơn, nhưng lại cải thiện độ chính xác và điểm BLEU.

6 Kết quả

6.1 Dịch máy

Trong bài kiểm tra dịch thuật Anh-Đức WMT 2014, mô hình Transformer lớn (Transformer (big) trong Bảng 2) vượt trội hơn các mô hình tốt nhất đã được báo cáo trước đây (bao gồm cả các mô hình kết hợp) với mức độ hơn 2.0 BLEU, thiết lập một điểm số BLEU tiên tiến mới. 28.4. Cấu hình của mô hình này được liệt kê ở dòng cuối cùng của Bảng 3. Quá trình huấn luyện đã diễn ra 3.5 ngày hôm nay 8 GPU P100. Ngay cả mô hình cơ bản của chúng tôi cũng vượt trội hơn tất cả các mô hình và mô hình kết hợp đã được công bố trước đây, với chi phí huấn luyện chỉ bằng một phần nhỏ so với bất kỳ mô hình cạnh tranh nào.

Trong bài kiểm tra dịch thuật Anh-Pháp WMT 2014, mô hình lớn của chúng tôi đạt được điểm BLEU là... 41.0, vượt trội hơn tất cả các mô hình đơn lẻ đã được công bố trước đây, với chi phí thấp hơn... 1/4 chi phí đào tạo của mô hình tiên tiến nhất trước đây. Mô hình Transformer (lớn) được đào tạo cho việc dịch từ tiếng Anh sang tiếng Pháp đã sử dụng tỷ lệ dropout. $P_{\text{làm rơi}}=0.1$, thay vì 0.3.

Đối với các mô hình cơ bản, chúng tôi sử dụng một mô hình duy nhất thu được bằng cách lấy trung bình 5 điểm kiểm tra cuối cùng, được ghi lại sau mỗi 10 phút. Đối với các mô hình lớn, chúng tôi lấy trung bình 20 điểm kiểm tra cuối cùng. Chúng tôi sử dụng thuật toán tìm kiếm chùm tia với kích thước chùm tia là 4 và hình phạt chiều dài $\alpha=0.6$ [38]. Các siêu tham số này được chọn sau khi thử nghiệm trên tập dữ liệu phát triển. Chúng tôi đặt độ dài đầu ra tối đa trong quá trình suy luận bằng độ dài đầu vào +50, nhưng chấm dứt sớm khi có thể [38].

Bảng 2 tóm tắt kết quả của chúng tôi và so sánh chất lượng dịch thuật cũng như chi phí đào tạo với các kiến trúc mô hình khác trong tài liệu. Chúng tôi ước tính số lượng phép toán dấu phẩy động được sử dụng để đào tạo mô hình bằng cách nhân thời gian đào tạo với số lượng GPU được sử dụng và ước tính khả năng xử lý dấu phẩy động đơn chính xác liên tục của mỗi GPU. 5.

6.2 Các biến thể mô hình

Để đánh giá tầm quan trọng của các thành phần khác nhau của Transformer, chúng tôi đã thay đổi mô hình cơ bản của mình theo nhiều cách khác nhau, đo lường sự thay đổi về hiệu suất trong quá trình dịch từ tiếng Anh sang tiếng Đức.

⁵Chúng tôi đã sử dụng các giá trị 2,8, 3,7, 6,0 và 9,5 TFLOPS lần lượt cho K80, K40, M40 và P100.

Bảng 3: Các biến thể của kiến trúc Transformer. Các giá trị không được liệt kê giống hệt với mô hình cơ bản. Tất cả các số liệu đều được thực hiện trên tập dữ liệu phát triển dịch thuật Anh-Đức, newstest2013. Độ phức tạp được liệt kê là trên mỗi từ ghép, theo mã hóa cặp byte của chúng tôi, và không nên so sánh với độ phức tạp trên mỗi từ.

	N	$d_{\text{người mẫu}}$	d_{ff}	h	d_k	d_v	$P_{\text{làm rơi}}$	ϵ_{ls}	xe lửa các bước	PPL (dev)	BLEU (dev)	tham số $\times 10^6$
căn cứ	6	512	2048	8	64	64	0,1	0,1	100K	4,92	25,8	65
(MỘT)										5,29	24,9	
										5,00	25,5	
										4,91	25,8	
										5,01	25,4	
(B)										5,16	25,1	58
										5,01	25,4	60
(C)	2											36
	4											50
	8											80
					32	32						28
					128	128						168
					1024	1024						53
(D)										5,77	24,6	
										4,95	25,5	
										4,67	25,3	
										5,47	25,7	
(E)	nhúng vị trí thay vì hình sin									4,92	25,7	
to lớn	6	1024	4096	16				0,3	300 nghìn	4,33	26,4	213

Bộ dữ liệu phát triển, newstest2013. Chúng tôi đã sử dụng thuật toán tìm kiếm chùm tia như mô tả trong phần trước, nhưng không sử dụng phương pháp trung bình hóa điểm kiểm tra. Chúng tôi trình bày các kết quả này trong Bảng 3.

Trong các hàng của Bảng 3 (A), chúng tôi thay đổi số lượng đầu chú ý và kích thước khóa và giá trị chú ý, giữ nguyên lượng tính toán, như mô tả trong Phần 3.2.2. Mặc dù cơ chế chú ý một đầu có chỉ số BLEU thấp hơn 0,9 so với thiết lập tốt nhất, chất lượng cũng giảm khi có quá nhiều đầu chú ý.

Trong các hàng của Bảng 3 (B), chúng ta quan sát thấy rằng việc giảm kích thước khóa chú ý làm tổn hại chất lượng mô hình. Điều này cho thấy việc xác định khả năng tương thích không dễ dàng và một hàm tương thích phức tạp hơn tích vô hướng có thể mang lại lợi ích. Chúng tôi tiếp tục quan sát ở hàng (C) và (D) rằng, như dự đoán, các mô hình lớn hơn sẽ tốt hơn và dropout rất hữu ích trong việc tránh quá khớp. Ở hàng (E), chúng tôi thay thế mã hóa vị trí hình sin của mình bằng các nhúng vị trí đã học [9] và quan sát thấy kết quả gần như giống hệt với mô hình cơ sở.

6.3 Phân tích cú pháp câu tiếng Anh

Để đánh giá xem Transformer có thể khái quát hóa sang các nhiệm vụ khác hay không, chúng tôi đã thực hiện các thí nghiệm về phân tích cú pháp thành phần tiếng Anh. Nhiệm vụ này đặt ra những thách thức cụ thể: đầu ra phải tuân theo các ràng buộc cấu trúc mạnh mẽ và dài hơn đáng kể so với đầu vào. Hơn nữa, các mô hình RNN chuỗi-đến-chuỗi chưa thể đạt được kết quả tiên tiến nhất trong các chế độ dữ liệu nhỏ [37].

Chúng tôi đã huấn luyện một mô hình transformer 4 lớp với $d_{\text{người mẫu}}=1024$ trên phần Wall Street Journal (WSJ) của Penn Treebank [25], khoảng 40K câu huấn luyện. Chúng tôi cũng huấn luyện nó trong môi trường bán giám sát, sử dụng các tập dữ liệu lớn hơn có độ tin cậy cao và BerkeleyParser với khoảng 17 triệu câu [37]. Chúng tôi đã sử dụng từ vựng gồm 16K mã thông báo cho môi trường chỉ WSJ và từ vựng gồm 32K mã thông báo cho môi trường bán giám sát.

Chúng tôi chỉ thực hiện một số ít thí nghiệm để lựa chọn dropout, cả attention và residual (mục 5.4), tốc độ học và kích thước chùm tia trên tập dữ liệu phát triển Mục 22, tất cả các tham số khác vẫn không thay đổi so với mô hình dịch cơ bản từ tiếng Anh sang tiếng Đức. Trong quá trình suy luận, chúng tôi

Bảng 4: Mô hình Transformer có khả năng khái quát hóa tốt việc phân tích cú pháp câu tiếng Anh (Kết quả được trình bày trong Mục 23 của WSJ)

Trình phân tích cú pháp	Đào tạo	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	Chỉ có WSJ, bài viết mang tính phân	88,3
Petrov et al. (2006) [29]	biệt đối xử Chỉ có WSJ, bài viết mang	90,4
Zhu et al. (2013) [40]	tính phân biệt đối xử Chỉ có WSJ, bài	90,4
Dyer et al. (2016) [8]	viết mang tính phân biệt đối xử	91,7
Biên tập (4 lớp)	Chỉ có WSJ mới đưa tin, mang tính phân biệt đối xử.	91,3
Zhu và cộng sự. (2013) [40]	bán giám sát	91,3
Huang & Harper (2009) [14]	bán giám sát	91,3
McClosky và cộng sự. (2006) [26]	bán giám sát	92.1
Vinyals & Kaiser et al. (2014) [37]	bán giám sát	92.1
Biên tập (4 lớp)	bán giám sát	92,7
Luong et al. (2015) [23]	đa nhiệm	93.0
Dyer et al. (2016) [8]	tạo sinh	93,3

Tăng chiều dài đầu ra tối đa lên bằng chiều dài đầu vào +300. Chúng tôi đã sử dụng kích thước chùm tia là 21 và $\alpha=0.3$. Áp dụng cho cả trường hợp chỉ sử dụng WSJ và trường hợp có giám sát một phần.

Kết quả của chúng tôi trong Bảng 4 cho thấy mặc dù thiếu điều chỉnh cụ thể cho nhiệm vụ, mô hình của chúng tôi hoạt động tốt một cách đáng ngạc nhiên, mang lại kết quả tốt hơn tất cả các mô hình đã được báo cáo trước đây, ngoại trừ Ngữ pháp Mạng thần kinh hồi quy [8].

Ngược lại với các mô hình RNN chuỗi-đến-chuỗi [37], Transformer vượt trội hơn Berkeley-Parser [29] ngay cả khi chỉ huấn luyện trên tập dữ liệu huấn luyện WSJ gồm 40K câu.

7. Kết luận

Trong công trình này, chúng tôi đã giới thiệu Transformer, mô hình chuyển đổi chuỗi đầu tiên hoàn toàn dựa trên cơ chế chú ý, thay thế các lớp mạng nơ-ron hồi quy thường được sử dụng trong kiến trúc mã hóa-giải mã bằng cơ chế tự chú ý đa đầu.

Đối với các tác vụ dịch thuật, Transformer có thể được huấn luyện nhanh hơn đáng kể so với các kiến trúc dựa trên các lớp mạng nơ-ron hồi quy hoặc mạng nơ-ron tích chập. Trên cả hai bài toán dịch tiếng Anh sang tiếng Đức và tiếng Anh sang tiếng Pháp trong kỳ thi WMT 2014, chúng tôi đã đạt được kết quả vượt trội mới. Trong bài toán dịch tiếng Anh sang tiếng Đức, mô hình tốt nhất của chúng tôi thậm chí còn vượt trội hơn tất cả các mô hình kết hợp đã được báo cáo trước đây.

Chúng tôi rất hào hứng về tương lai của các mô hình dựa trên cơ chế chú ý và dự định áp dụng chúng vào các nhiệm vụ khác. Chúng tôi dự định mở rộng Transformer cho các bài toán liên quan đến các phương thức đầu vào và đầu ra khác ngoài văn bản và nghiên cứu các cơ chế chú ý cục bộ, hạn chế để xử lý hiệu quả các đầu vào và đầu ra lớn như hình ảnh, âm thanh và video. Giảm tính tuần tự trong quá trình tạo ra dữ liệu cũng là một mục tiêu nghiên cứu khác của chúng tôi.

Mã nguồn chúng tôi sử dụng để huấn luyện và đánh giá mô hình có sẵn tại đây: <https://github.com/tensorflow/tensor2tensor>.

Lời cảm ơn Chúng tôi xin bày tỏ lòng biết ơn đến Nal Kalchbrenner và Stephan Gouws vì những nhận xét, chỉnh sửa và nguồn cảm hứng hữu ích của họ.

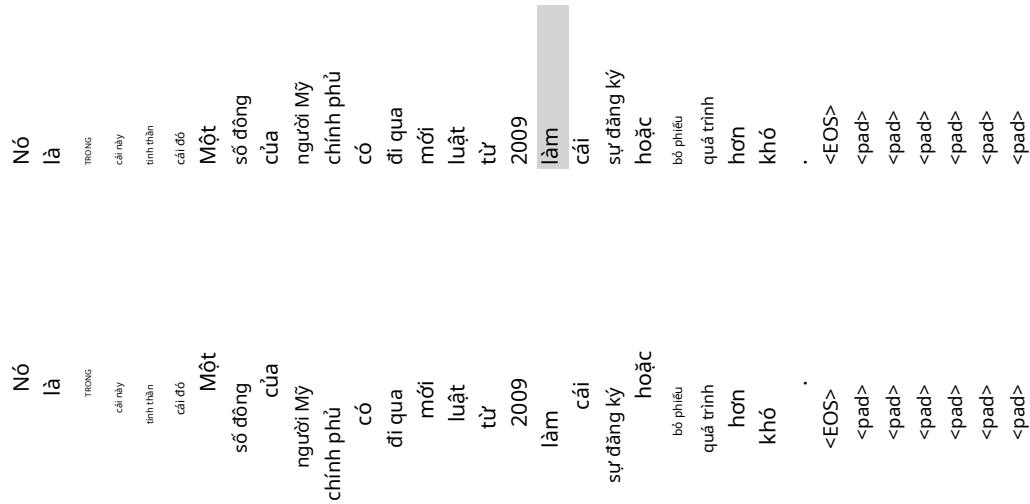
Tài liệu tham khảo

- [1] Jimmy Lei Ba, Jamie Ryan Kiros và Geoffrey E Hinton. Chuẩn hóa lớp. *bản in trước arXiv arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho và Yoshua Bengio. Dịch máy thần kinh bằng cách học đồng thời căn chỉnh và dịch. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong và Quoc V. Le. Khám phá quy mô lớn về kiến trúc dịch máy thần kinh. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong và Mirella Lapata. Mạng bộ nhớ ngắn hạn dài hạn cho việc đọc máy. *bản in trước arXiv arXiv:1601.06733*, 2016.

- [5] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gulcehre, Fethi Bougares, Holger Schwenk và Yoshua Bengio. Học biểu diễn cụm từ bằng cách sử dụng bộ mã hóa-giải mã rnn cho dịch máy thống kê. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Học sâu với các phép tích chập phân tách theo chiều sâu. *bản in trước arXiv arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho và Yoshua Bengio. Đánh giá thực nghiệm về mạng nơ-ron hồi quy có cổng trên mô hình chuỗi. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros và Noah A. Smith. Ngữ pháp mạng thần kinh hồi quy. Trong *Biên bản của NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats và Yann N. Dauphin. Học tập chuỗi tích chập từ chuỗi này sang chuỗi khác. *Bản thảo arXiv arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Tạo chuỗi bằng mạng thần kinh hồi quy. *bản in trước arXiv arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren và Jian Sun. Học dư sâu để nhận dạng hình ảnh. TRONG *Kỷ yếu Hội nghị IEEE về Thị giác máy tính và Nhận dạng mẫu*, trang 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi và Jürgen Schmidhuber. Luồng gradient trong mạng nơ-ron hồi quy: khó khăn trong việc học các phụ thuộc dài hạn, 2001.
- [13] Sepp Hochreiter và Jürgen Schmidhuber. Trí nhớ ngắn hạn dài. *Tính toán thần kinh*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang và Mary Harper. Tự huấn luyện ngữ pháp PCFG với chú thích tiềm ẩn trên nhiều ngôn ngữ. Trong *Kỷ yếu Hội nghị về Phương pháp Thực nghiệm trong Xử lý Ngôn ngữ Tự nhiên năm 2009*, trang 832–841. ACL, tháng 8 năm 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer và Yonghui Wu. Khám phá giới hạn của mô hình ngôn ngữ. *bản in trước arXiv arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser và Samy Bengio. Liệu trí nhớ chủ động có thể thay thế sự chú ý không? Trong *Những tiến bộ trong hệ thống xử lý thông tin thần kinh (NIPS)*, 2016.
- [17] Łukasz Kaiser và Ilya Sutskever. GPU thần kinh học các thuật toán. Trong *Hội nghị quốc tế về biểu diễn học tập (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves và Koray Kavukcuoglu. Dịch máy thần kinh trong thời gian tuyến tính. *Bản thảo arXiv arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang và Alexander M. Rush. Mạng lưới chú ý có cấu trúc. Trong *Hội nghị quốc tế về biểu diễn học tập*, 2017.
- [20] Diederik Kingma và Jimmy Ba. Adam: Một phương pháp tối ưu hóa ngẫu nhiên. Trong *ICLR*, 2015.
- [21] Oleksii Kuchaiev và Boris Ginsburg. Thủ thuật phân tích nhân tử cho mạng LSTM. *bản in trước arXiv arXiv:1703.10722*, 2017.
- [22] Chu Hàn Lâm, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Chu, và Yoshua Bengio. Một câu nhúng có cấu trúc chú ý đến bản thân. *bản in trước arXiv arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals và Łukasz Kaiser. Học chuỗi đa nhiệm vụ. *bản in trước arXiv arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham và Christopher D Manning. Các phương pháp hiệu quả để dịch máy thần kinh dựa trên sự chú ý. *bản in trước arXiv arXiv:1508.04025*, 2015.

- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz và Beatrice Santorini. Xây dựng một kho ngữ liệu chú thích lớn của tiếng Anh: Penn Treebank. *Ngôn ngữ học tính toán*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak và Mark Johnson. Tự đào tạo hiệu quả để phân tích cú pháp. Trong *Kỷ yếu Hội nghị Công nghệ Ngôn ngữ Nhân văn của NAACL, Hội nghị chính*, trang 152–159. ACL, tháng 6 năm 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, và Jakob Uszkoreit. Một mô hình chú ý có thể phân hủy. TRONG *Các phương pháp thực nghiệm trong xử lý ngôn ngữ tự nhiên*, 2016.
- [28] Romain Paulus, Caiming Xiong và Richard Socher. Một mô hình được củng cố sâu sắc cho tóm tắt trừu tượng. *bản in trước arXiv arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux và Dan Klein. Học cách chú thích cây chính xác, gọn nhẹ và dễ hiểu. Trong *Kỷ yếu Hội nghị Quốc tế lần thứ 21 về Ngôn ngữ học Tính toán và Hội nghị Thường niên lần thứ 44 của ACL*, trang 433–440. ACL, tháng 7 năm 2006.
- [30] Ofir Press và Lior Wolf. Sử dụng nhúng đầu ra để cải thiện mô hình ngôn ngữ. *bản in trước arXiv arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow và Alexandra Birch. Dịch máy thần kinh của các từ hiếm với các đơn vị dưới từ. *bản in trước arXiv arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton và Jeff Dean. Mạng nơ-ron cực lớn: Lớp hỗn hợp chuyên gia cổng thưa. *bản in trước arXiv arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever và Ruslan Salakhutdinov. Dropout: một cách đơn giản để ngăn chặn mạng nơ-ron bị quá khớp. *Tạp chí nghiên cứu học máy*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston và Rob Fergus. Mạng bộ nhớ đầu cuối. Trong C. Cortes, ND Lawrence, DD Lee, M. Sugiyama và R. Garnett, các biên tập viên, *Những tiến bộ trong hệ thống xử lý thông tin thần kinh 28*, trang 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals và Quoc VV Le. Học trình tự với mạng nơ-ron. Trong *Những tiến bộ trong hệ thống xử lý thông tin thần kinh*, trang 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens và Zbigniew Wojna. Suy nghĩ lại về kiến trúc khởi đầu cho thị giác máy tính. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, và Hinton. Ngữ pháp như một ngoại ngữ. Trong *Những tiến bộ trong hệ thống xử lý thông tin thần kinh*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quốc V Lê, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Hệ thống dịch máy thần kinh của Google: Thu hẹp khoảng cách giữa bản dịch của con người và máy. *bản in trước arXiv arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li và Wei Xu. Mô hình hồi quy sâu với kết nối tiến nhanh cho dịch máy thần kinh. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang và Jingbo Zhu. Phân tích thành phần giảm dịch chuyển nhanh và chính xác. TRONG *Kỷ yếu Hội nghị thường niên lần thứ 51 của ACL (Tập 1: Các bài báo dài)*, trang 434–443. ACL, tháng 8 năm 2013.

Hình ảnh hóa sự chú ý



Hình 3: Một ví dụ về cơ chế chú ý theo dõi các phụ thuộc khoảng cách xa trong cơ chế tự chú ý của bộ mã hóa ở lớp 5 trong tổng số 6 lớp. Nhiều đầu chú ý tập trung vào một phụ thuộc xa của động từ 'làm', hoàn thành cụm từ 'làm cho...khó khăn hơn'. Sự chú ý ở đây chỉ được hiển thị cho từ 'làm'. Các màu khác nhau đại diện cho các đầu chú ý khác nhau. Nên xem ở chế độ màu.

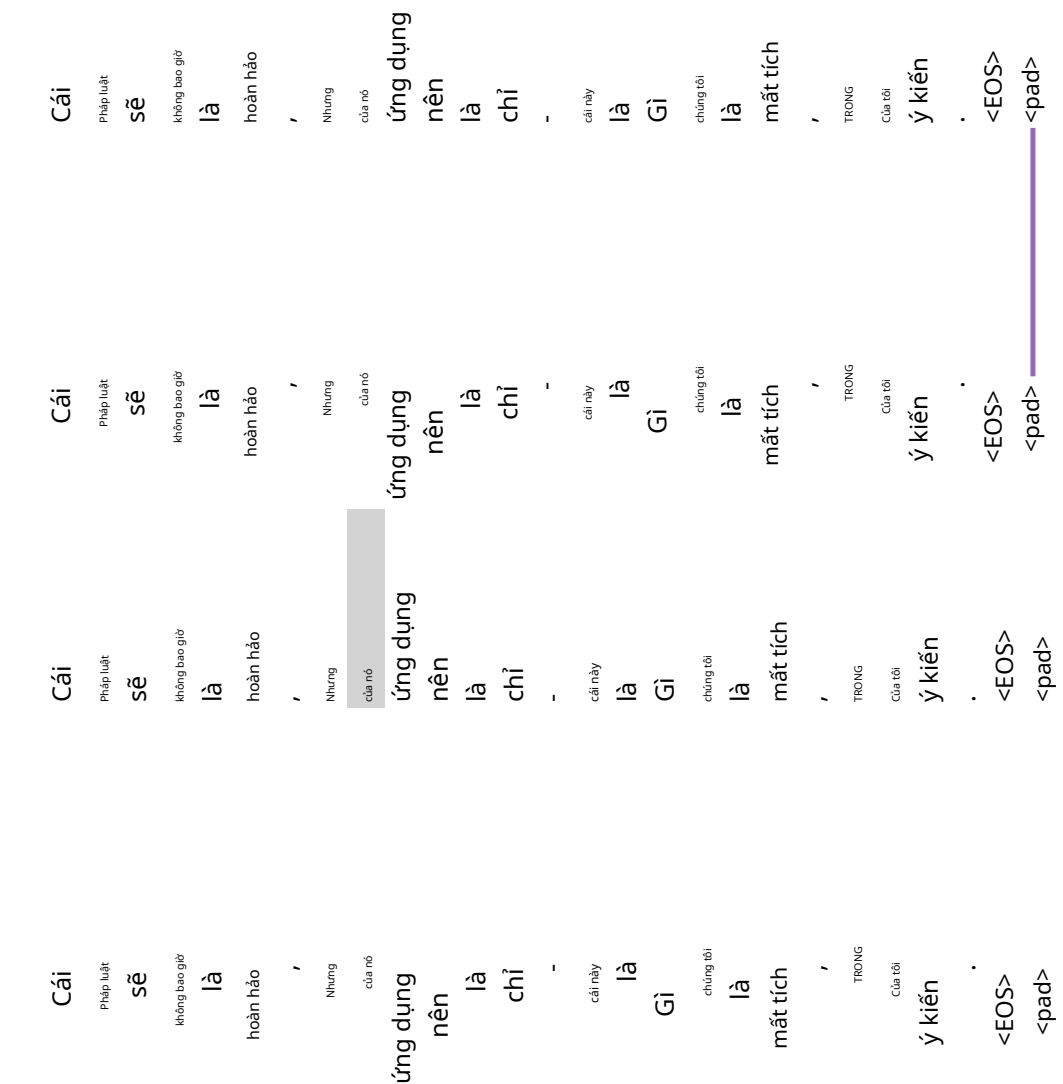


Figure 4: Twaett ntiTRÊN S. layer5 o 6 dùng như liên quan đến việc giải quyết phép thế đại từ. Trên cùng: Đây attention vì Anh quảng 5. Bottom: Là related từ chỉ từ từ 'its' để thu hút sự chú ý, 5 từ s. 16. Note đó haett ntiTRÊN S apf hoặc

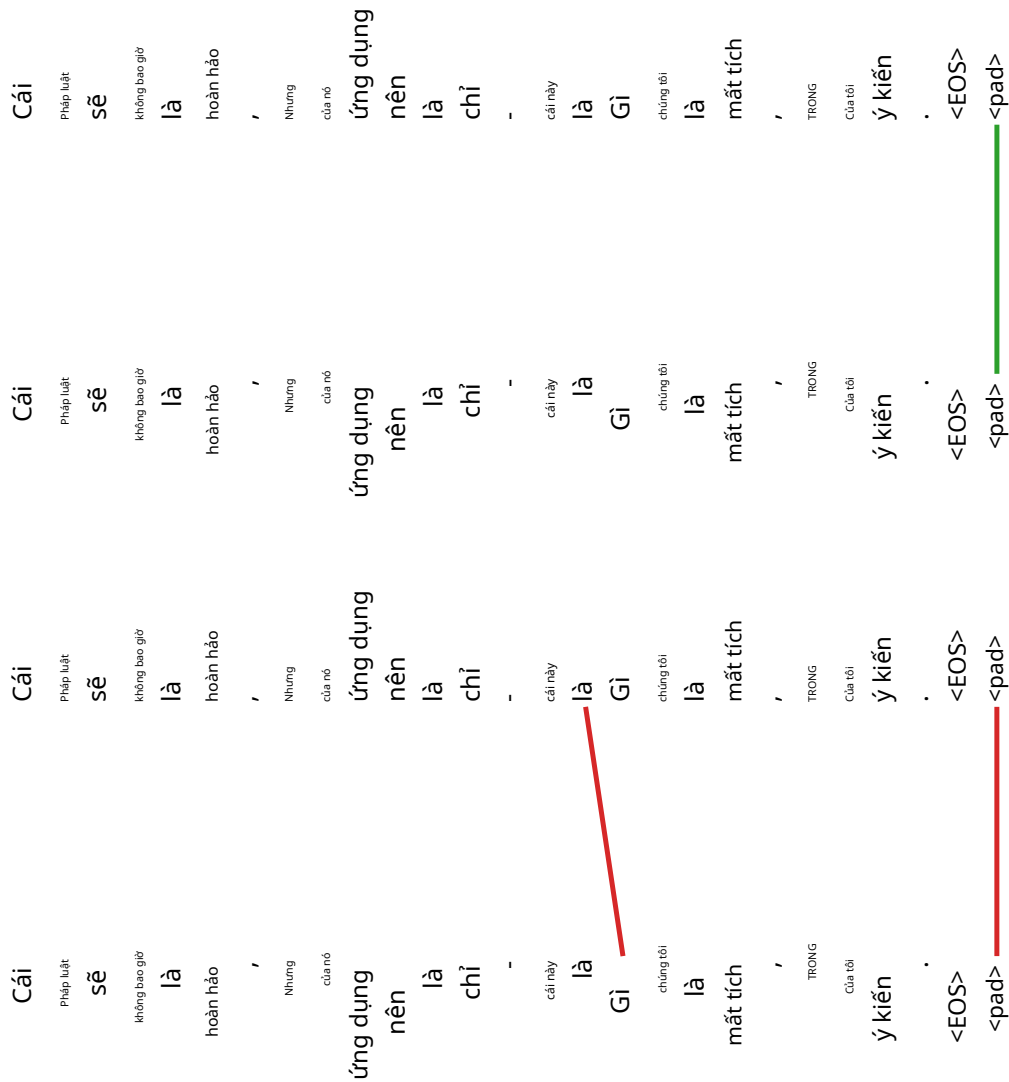


Figure 5: Nhiều người tập trung chú ý thể hiện hành vi dường như có liên quan đến cấu trúc của...
Sentence. We give hai đầu ex-Moties ab ve, from Hai đầu khác nhau từ cơ chế tự chú ý của bộ mã hóa trong các
Một vòng của 6. The heads clear và Đĩa khác nhau khác nhau.